

LAPORAN PRAKTIKUM 3
PEMOGRAMAN BERORIENTASI OBJEK

Disusun Oleh:

Dinda Amelia

(2511531020)

Dosen Pengampu:

Nurfiah, S.ST, M.Kom.

Asisten Praktikum:

Ezza Addini



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
PADANG 2026

KATA PENGANTAR

Segala puji bagi Allah SWT yang telah memberikan kesempatan serta kemudahan kepada penulis untuk dapat menyelesaikan Laporan Praktikum pada mata kuliah Pemrograman Berorientasi Objek ini tepat waktu. Sholawat serta salam juga penulis sampaikan kepada baginda tercinta Nabi Muhammad SAW.

Laporan praktikum ini disusun sebagai bentuk pertanggungjawaban atas pelaksanaan kegiatan praktikum Pemrograman Berorientasi Objek Modul 3 yang membahas tentang *Information Hiding*, *Access Modifiers* (private, public, protected), pembatasan akses data melalui *Getter*, serta penerapan Enkapsulasi pada validasi *business logic* dan autentikasi PIN.

Dalam penyusunan Laporan Praktikum ini, penulis menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun demi kesempurnaan laporan ini.

Padang, 2026

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I.....	1
PENDAHULUAN	1
1.1 Latar Belakang.....	1
1.2 Tujuan	1
1.3 Manfaat	2
1.4 Link repository Github:.....	2
BAB II.....	3
PEMBAHASAN	3
2.1 Dasar Teori.....	3
2.2 Langkah kerja	4
2.2.1 Class 1 “Transaksi”	4
2.2.2 Class 2 “Rekening”	5
2.2.3 Class 3 “Main”	7
2.3 Tugas Praktikum: Mengamankan CLI dengan Otentikasi	9
BAB III	12
KESIMPULAN.....	12

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam pengembangan perangkat lunak berbasis pemrograman berorientasi objek (PBO), keamanan data dan integritas *state* suatu objek merupakan faktor yang sangat krusial. Pada perancangan aplikasi dasar, atribut kelas sering kali dibuat tanpa pembatas visibilitas (*default modifier*), yang memungkinkan variabel internal diakses dan diubah secara bebas langsung dari luar kelas. Praktik semacam ini berpotensi menimbulkan manipulasi data yang berbahaya, seperti pengubahan saldo rekening tanpa melalui prosedur transaksi resmi.

Untuk mengatasi permasalahan tersebut, diterapkan prinsip *Information Hiding* dan Enkapsulasi. Konsep ini menyembunyikan detail implementasi internal dan melindungi variabel peka dengan menentukan batasan akses melalui *access modifiers* (seperti *private*) serta hanya membuka antarmuka terontrol berupa method *getter*. Praktikum Modul 3 ini memberikan pemahaman mendalam bagi mahasiswa untuk mengimplementasikan enkapsulasi guna melindungi entitas transaksi dan rekening, serta menyisipkan validasi *business logic* seperti autentikasi PIN pada alur program.

1.2 Tujuan

1. Memahami urgensi *Information Hiding* dalam rekayasa perangkat lunak.
2. Mengimplementasikan *Access Modifiers* (*private*, *public*, *protected*) dengan tepat.
3. Mengontrol akses pembacaan dan penulisan atribut menggunakan method *Getter*.

1.3 Manfaat

1. Memberikan pemahaman praktis mengenai mekanisme perlindungan data sensitif dari manipulasi langsung.
2. Melatih keterampilan dalam menyusun arsitektur kelas yang aman, terstruktur, dan *immutable* bilamana diperlukan.
3. Menjadi landasan utama dalam merancang sistem perbankan mini yang mengedepankan otorisasi akses dan validasi data.

1.4 Link repository Github:

1. https://github.com/faridaarifin91-sudo/PrakPBO_B_2511531020/tree/Pratikum-3/PrakPBO

BAB II

PEMBAHASAN

2.1 Dasar Teori

Dalam Pemrograman Berorientasi Objek (PBO), Enkapsulasi dan *Information Hiding* merupakan pilar utama untuk menjaga keamanan serta integritas data objek:

- *Information Hiding*: Robert C. Martin (2008) menyatakan bahwa detail implementasi internal suatu kelas harus disembunyikan. Objek seharusnya mengekspos *behavior* (method), bukan mengekspos data (variabel) secara langsung.
- *Access Modifiers* di Java: Kata kunci untuk mengatur tingkat visibilitas akses:
 - *private*: Akses paling ketat, variabel/method hanya dapat diakses dari dalam kelas itu sendiri. Aturan utama untuk mengunci atribut.
 - *protected*: Akses menengah, dapat diakses oleh kelas dalam *package* yang sama dan *subclass*.
 - *public*: Akses paling terbuka, dapat diakses dari mana saja. Dipakai untuk method layanan luar.
- *Getter* dan *Setter*: Menurut Oracle (2023), pengaksesan variabel *private* harus difasilitasi lewat method *public* khusus. Melalui *getter/setter*, program dapat menyisipkan validasi seperti cek format PIN sebelum data diproses.

2.2 Langkah kerja

1. jalankan aplikasi Eclipse IDE pada komputer/laptop.



2. Buka project PrakPBO_B_2511531020 dan pastikan berada dalam package Pekan3.

2.2.1 Class 1 “Transaksi”

1. Kode Program

```
PrakPBO_B_2511531020 > src > pekan3 > Transaksi > getNominal() : double
1 package pekan3;
2
3 public class Transaksi {
4     //1. Mengubah semua atribut menjadi private
5     private String idTransaksi;
6     private String jenis;
7     private double nominal;
8
9     public Transaksi(String id, String jenis, double nominal) {
10         this.idTransaksi = id;
11         this.jenis = jenis;
12         this.nominal = nominal;
13     }
14     //2. Hanya menyediakan Getter (Read-only)
15     public String getIdTransaksi() {return idTransaksi;}
16     public String getJenis() {return jenis;}
17     public double getNominal() {return nominal;}
18
19     public void cetakDetail() {
20         System.out.println("ID: " + idTransaksi + " | Jenis: " + jenis + " | Nominal: Rp" + nominal);
21     }
22 }
```

2. Analisis Langkah

a. Struktur Program

- Berfungsi sebagai model entitas data riwayat transaksi yang bersifat *immutable* (hanya bisa dibaca/*read-only* setelah diinstansiasi).

b. Elemen Sintaks Utama

- `private String idTransaksi, jenis, nominal` → atribut dikunci menjadi `private` agar tidak dapat dimanipulasi dari luar.

- `public Transaksi(...)` → *constructor* untuk menginisialisasi atribut transaksi.
 - Method *Getter* (`getIdTransaksi()`, `getJenis()`, `getNominal()`) → menyediakan akses pembacaan nilai atribut.
 - `cetakDetail()` → method untuk menampilkan rincian riwayat transaksi ke konsol.
- c. Alur Kerja Sintaks
- Saat objek diinstansiasi lewat *constructor*, data transaksi dikunci.
 - Karena tidak disediakan method *setter*, nilai transaksi tidak dapat diubah kembali.
 - Akses data dari luar hanya dilayani oleh method *getter* atau pencetakan ringkasan via `cetakDetail()`.

2.2.2 Class 2 “Rekening”

1. Kode Program

```

PrakPBO_B_2511531020 > src > pekan3 > Rekening >
1 package pekan3;
2
3 import java.util.ArrayList;
4
5 public class Rekening {
6     // 1. Mengunci atribut sensitif
7     private String nomorRekening;
8     private String namaPemilik;
9     private double saldo;
10    private String pin; // Data sensitif
11    private ArrayList<Transaksi> riwayatTransaksi;
12
13    // 2. Constructor dengan validasi PIN
14    public Rekening(String nomor, String nama, double saldoAwal, String pinAwal) {
15        this.nomorRekening = nomor;
16        this.namaPemilik = nama;
17        this.saldo = saldoAwal;
18
19        // Validasi PIN 6 digit
20        if (pinAwal.length() == 6) {
21            this.pin = pinAwal;
22        } else {
23            System.out.println("Peringatan: PIN harus 6 digit! Menggunakan PIN default 123456");
24            this.pin = "123456";
25        }
26
27        this.riwayatTransaksi = new ArrayList<>();
28        System.out.println("Rekening atas nama " + namaPemilik + " berhasil dibuat.");
29    }
30
31    // 3. Getter terfilter
32    public String getNomorRekening() { return nomorRekening; }
33    public String getNamaPemilik() { return namaPemilik; }
34    public double getSaldo() { return saldo; }

```



```

36 // 4. Method Otentikasi
37 public boolean otentikasi(String inputPin) {
38     return this.pin.equals(inputPin);
39 }
40
41 public void setorTunai(double nominal) {
42     if (nominal > 0) {
43         saldo += nominal;
44         riwayatTransaksi.add(new Transaksi("TRX-S-" + System.currentTimeMillis(), "Kredit", nominal));
45         System.out.println("Setor tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
46     } else {
47         System.out.println("Gagal: nominal setor harus lebih dari 0!");
48     }
49 }
50
51 public void tarikTunai(double nominal) {
52     if (nominal < 10000) {
53         System.out.println("Transaksi Gagal: Minimal nominal penarikan 10.000");
54     } else if (nominal > saldo) {
55         System.out.println("Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp" + saldo);
56     } else {
57         saldo -= nominal;
58         riwayatTransaksi.add(new Transaksi("TRX-T-" + System.currentTimeMillis(), "Debit", nominal));
59         System.out.println("Tarik tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
60     }
61 }
62
63 public void cekInformasi() {
64     System.out.println("--- INFO REKENING ---");
65     System.out.println("No. Rekening : " + nomorRekening);
66     System.out.println("Nama Pemilik : " + namaPemilik);
67     System.out.println("Saldo Akhir : Rp" + saldo);
68     System.out.println("-----");
69 }
70
71 public void cetakMutasi() {
72     System.out.println("\n===== MUTASI REKENING =====");
73     System.out.println("No. Rekening : " + nomorRekening);
74     System.out.println("Nama Pemilik : " + namaPemilik);
75     System.out.println("-----");
76     if (riwayatTransaksi.isEmpty()) {
77         System.out.println("Belum ada riwayat transaksi.");
78     } else {
79         for (Transaksi t : riwayatTransaksi) {
80             t.cetakDetail();
81         }
82     }
83     System.out.println("-----");
84 }
85 }

```

2. Analisis Langkah

a. Struktur Program

- Bertindak sebagai entitas pengelola rekening yang mengamankan saldo dan enkapsulasi akses PIN.

b. Elemen Sintaks Utama

- `private String nomorRekening, namaPemilik, saldo, pin` → data sensitif disembunyikan menggunakan modifier `private`.
- `public Rekening(...)` → *constructor* dengan logika validasi panjang PIN.
- `otentikasi(String inputPin)` → method *public* berbasis boolean untuk mengecek kebenaran PIN tanpa membocorkan nilai variabel `pin`.
- Method *Getter* (`getNomorRekening()`, `getNamaPemilik()`, `getSaldo()`) → pengaksesan data secara terbatas. Method `getPin()` dan `setSaldo()` sengaja ditiadakan demi keamanan.

c. Alur Kerja Sintaks

- Saat *constructor* dipanggil, sistem memeriksa panjang string pinAwal. Jika pas 6 digit, PIN disimpan; jika tidak, muncul pesan peringatan dan dialokasikan ke PIN default "123456".
- Panggilan otentikasi(inputPin) membandingkan masukan PIN menggunakan equals() dan mengembalikan respon true atau false.

2.2.3 Class 3 “Main”

1. Kode Program

```

1 package pekan3;
2
3 import java.util.ArrayList;
4 import java.util.Scanner;
5
6 public class Main {
7     public static void main(String[] args) {
8         Scanner input = new Scanner(System.in);
9         ArrayList<Rekening> daftarRekening = new ArrayList<>();
10        Rekening akunAktif = null;
11        boolean isRunning = true;
12
13        System.out.println("=== SISTEM PERBANKAN MINI (ENKAPSULASI) ===");
14
15        while (isRunning) {
16            System.out.println("\nMenu Utama:");
17            System.out.println("1. Buka Rekening Baru");
18            System.out.println("2. Setor Tunai");
19            System.out.println("3. Tarik Tunai (Wajib PIN)");
20            System.out.println("4. Cek Informasi Rekening");
21            System.out.println("5. Ganti Akun");
22            System.out.println("6. Cetak Mutasi (Wajib PIN)");
23            System.out.println("0. Keluar");
24            System.out.print("Pilih menu: ");
25            int pilihan = input.nextInt();
26            input.nextLine(); // Clear buffer
27
28            switch (pilihan) {
29                case 1:
30                    System.out.print("Masukkan No Rekening: ");
31                    String no = input.nextLine();
32                    System.out.print("Masukkan Nama Pemilik: ");
33                    String nama = input.nextLine();
34                    System.out.print("Masukkan Saldo Awal: ");

```

```

35     double saldo = input.nextDouble();
36     input.nextLine();
37     System.out.print("Masukkan PIN Baru (6 digit): ");
38     String pinBaru = input.nextLine();
39
40     Rekening akunBaru = new Rekening(no, nama, saldo, pinBaru);
41     daftarRekening.add(akunBaru);
42     akunAktif = akunBaru;
43     break;
44
45 case 2:
46     if (akunAktif == null) {
47         System.out.println("Error: Anda belum memilih/memiliki rekening!");
48     } else {
49         System.out.print("Masukkan nominal setor: ");
50         double setor = input.nextDouble();
51         akunAktif.setorTunai(setor);
52     }
53     break;
54
55 case 3:
56     if (akunAktif == null) {
57         System.out.println("Error: Anda belum memilih/memiliki rekening!");
58     } else {
59         System.out.print("Masukkan PIN Anda: ");
60         String pinInput = input.nextLine();
61         if (akunAktif.otentikasi(pinInput)) {
62             System.out.print("Masukkan nominal tarik tunai: ");
63             double tarik = input.nextDouble();
64             akunAktif.tarikTunai(tarik);
65         } else {
66             System.out.println("Akses Ditolak: PIN yang Anda masukkan salah!");
67         }
68     }
69     break;
70
71 case 4:
72     if (akunAktif == null) {
73         System.out.println("Error: Anda belum memiliki rekening!");
74     } else {
75         akunAktif.cekInformasi();
76     }
77     break;
78
79 case 5:
80     if (daftarRekening.isEmpty()) {
81         System.out.println("Belum ada rekening yang terdaftar.");
82     } else {
83         System.out.print("Masukkan Nomor Rekening yang ingin diaktifkan: ");
84         String cariNo = input.nextLine();
85         boolean ditemukan = false;
86         for (Rekening r : daftarRekening) {
87             if (r.getNomorRekening().equalsIgnoreCase(cariNo)) {
88                 akunAktif = r;
89                 ditemukan = true;
90                 System.out.println("Berhasil mengganti ke akun milik: " + r.getNamaPemilik());
91                 break;
92             }
93         }
94         if (!ditemukan) {
95             System.out.println("Error: Nomor rekening tidak ditemukan!");
96         }
97     }
98     break;
99
100 case 6:
101     if (akunAktif == null) {
102         System.out.println("Error: Anda belum memiliki rekening!");
103     } else {
104         System.out.print("Masukkan PIN Anda: ");
105         String pinInput = input.nextLine();
106         if (akunAktif.otentikasi(pinInput)) {
107             akunAktif.cetakMutasi();
108         } else {
109             System.out.println("Akses Ditolak: PIN yang Anda masukkan salah!");
110         }
111     }
112     break;
113
114 case 0:
115     isRunning = false;
116     System.out.println("Sistem ditutup. Terima kasih!");
117     break;
118
119 default:
120     System.out.println("Pilihan tidak valid!");
121 }
122 }
123 input.close();
124 }
125 }

```

2. Analisis Langkah

a. Struktur Program

- Kelas Main berada dalam *package* Pekan1.
- Berfungsi sebagai penggerak CLI (*Command Line Interface*) simulasi bank mini berbasis menu.

b. Elemen Sintaks Utama

- `ArrayList<Rekening> daftarRekening` → wadah koleksi dinamis.
- `Rekening akunAktif` → menunjuk ke objek rekening yang sedang aktif.
- Scanner input → membaca konsol.
- switch-case → kontrol alur navigasi menu.
- `akunAktif.otentikasi(pinInput)` → lapisan gerbang keamanan.

c. Alur Kerja Sintaks

- Case 1: Meminta data registrasi akun termasuk masukan PIN 6 digit.
- Case 3 & 6: Sebelum memproses tarik tunai atau cetak mutasi, sistem mewajibkan verifikasi PIN via `otentikasi()`.
- Case 5: Menggunakan *getter* (`r.getNomorRekening()`) untuk melakukan pencarian di `ArrayList`.

2.3 Tugas Praktikum: Mengamankan CLI dengan Otentikasi

1. Pendaftaran Rekening Baru dengan Input PIN 6 Digit:

- File `Main.java` (Baris 37–38): Meminta masukan PIN 6 digit dari pengguna saat membuat akun baru.

```
37         System.out.print("Masukkan PIN Baru (6 digit): ");
38         String pinBaru = input.nextLine();
```

- File `Rekening.java` (Baris 19–25): Mengecek apakah panjang string PIN pas 6 digit. Jika tidak valid, sistem menetapkan PIN default "123456".

```
19         // Validasi PIN 6 digit
20         if (pinAwal.length() == 6) {
21             this.pin = pinAwal;
22         } else {
23             System.out.println("Peringatan: PIN harus 6 digit! Menggunakan PIN default 123456");
24             this.pin = "123456";
25         }
```

2. Pengamanan Fitur Tarik Tunai (Menu 3) dengan Otentikasi PIN:

- File Main.java (Baris 59–67): Sebelum memproses penarikan uang, sistem meminta PIN dan memanggil method otentikasi().

```
59         System.out.print("Masukkan PIN Anda: ");
60         String pinInput = input.nextLine();
61         if (akunAktif.otentikasi(pinInput)) {
62             System.out.print("Masukkan nominal tarik tunai: ");
63             double tarik = input.nextDouble();
64             akunAktif.tarikTunai(tarik);
65         } else {
66             System.out.println("Akses Ditolak: PIN yang Anda masukkan salah!");
67         }
```

3. Pengamanan Fitur Cetak Mutasi (Menu 6) dengan Otentikasi PIN:

- File Main.java (Baris 104–110): Verifikasi PIN diwajibkan sebelum riwayat transaksi dapat ditampilkan.

```
104         System.out.print("Masukkan PIN Anda: ");
105         String pinInput = input.nextLine();
106         if (akunAktif.otentikasi(pinInput)) {
107             akunAktif.cetakMutasi();
108         } else {
109             System.out.println("Akses Ditolak: PIN yang Anda masukkan salah!");
110         }
```

4. Tampilan Output Eksekusi Program:

- Output Buka Rekening dengan PIN Valid (6 Digit):

```
Main (3) [Java Application] C:\Users\USER\.p2\pool\plugins\org.eclipse.justj.openjdk.hot
2. Setor Tunai
3. Tarik Tunai (Wajib PIN)
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Wajib PIN)
0. Keluar
Pilih menu: 1
Masukkan No Rekening: 101002
Masukkan Nama Pemilik: Dinda Amelia
Masukkan Saldo Awal: 500000
Masukkan PIN Baru (6 digit): 654321
Rekening atas nama Dinda Amelia berhasil dibuat.
```

- Output Buka Rekening dengan PIN Tidak Valid (< 6 Digit):

```
Pilih menu: 1
Masukkan No Rekening: 101003
Masukkan Nama Pemilik: Dika
Masukkan Saldo Awal: 200000
Masukkan PIN Baru (6 digit): 123
Peringatan: PIN harus 6 digit! Menggunakan PIN default 123456
Rekening atas nama Dika berhasil dibuat.
```

- Output Tarik Tunai dengan PIN Salah (Akses Ditolak):

```
Main (3) [Java Application] C:\Users\USER\.p2\pool\plugins\org.eclipse
```

```
Menu Utama:  
1. Buka Rekening Baru  
2. Setor Tunai  
3. Tarik Tunai (Wajib PIN)  
4. Cek Informasi Rekening  
5. Ganti Akun  
6. Cetak Mutasi (Wajib PIN)  
0. Keluar  
Pilih menu: 3  
Masukkan PIN Anda: 111111  
Akses Ditolak: PIN yang Anda masukkan salah!
```

- Output Tarik Tunai & Cetak Mutasi dengan PIN Benar:

```
Menu Utama:  
1. Buka Rekening Baru  
2. Setor Tunai  
3. Tarik Tunai (Wajib PIN)  
4. Cek Informasi Rekening  
5. Ganti Akun  
6. Cetak Mutasi (Wajib PIN)  
0. Keluar  
Pilih menu: 3  
Masukkan PIN Anda: 654321  
Masukkan nominal tarik tunai: 50000  
Tarik tunai Rp50000.0 berhasil. Saldo saat ini: Rp450000.0  
  
Menu Utama:  
1. Buka Rekening Baru  
2. Setor Tunai  
3. Tarik Tunai (Wajib PIN)  
4. Cek Informasi Rekening  
5. Ganti Akun  
6. Cetak Mutasi (Wajib PIN)  
0. Keluar  
Pilih menu: 6  
Masukkan PIN Anda: 654321  
  
===== MUTASI REKENING =====  
No. Rekening : 101002  
Nama Pemilik : Dinda Amelia  
-----  
ID: TRX-T-1790520046194 | Jenis: Debit | Nominal: Rp50000.0  
-----
```

BAB III

KESIMPULAN

Praktikum Modul 3 ini berhasil mengimplementasikan prinsip *Information Hiding* dan Enkapsulasi pada simulasi sistem perbankan mini. Pengubahan visibilitas seluruh atribut kelas—khususnya pada Rekening dan Transaksi—menjadi private terbukti efektif mengamankan variabel sensitif seperti saldo dan pin dari manipulasi langsung di luar kelas. Penggunaan method *Getter* terkonfirmasi mampu membatasi akses pembacaan data secara terukur, sementara ketiadaan method *Setter* pada entitas Transaksi secara sukses mewujudkan sifat data yang *immutable*.

Pengintegrasian validasi *business logic* dan otentikasi PIN pada kelas Main juga berhasil meningkatkan keamanan antarmuka CLI pada transaksi kritis. Sistem secara otomatis menetapkan PIN *default* apabila masukan pengguna tidak memenuhi kriteria 6 digit saat pendaftaran akun. Selain itu, pemanggilan method otentikasi() pada fitur Tarik Tunai dan Cetak Mutasi terbukti secara akurat mampu memvalidasi hak akses, di mana eksekusi program langsung ditolak jika PIN yang dimasukkan tidak sesuai.